

Report conversation

js

```
const firstLetter = this.selectedCreator
displayedCreator =
  this.model.creators.find(creator =>
    creator.toUpperCase().startsWith(1
  ) ?? displayedCreator; // Fall back
```

This has a bug, where "Émile" is not selected as the displayed creator when "E" is the selected filter letter.

The problem seems to be related to the `toUpperCase()` method and the handling of diacritical marks. JavaScript's `toUpperCase()` method doesn't consider diacritical marks, so when you transform "Émile" to uppercase, you get "ÉMILE", not "EMILE". As a result, when you check if "ÉMILE".startsWith("E"), you get `false`.

To solve this problem, you can normalize the string to remove diacritical marks using the `normalize()` method with the "NFD" form, which splits an accented letter into the ASCII letter and a separate diacritic. You can then remove the diacritics using a regular expression that matches all non-ASCII characters. Here's how you can do it:

js

 Copy code

```
const firstLetter = this.selectedCreatorFilter.toUpperCase();

displayedCreator =
  this.model.creators.find(creator =>
    creator.normalize("NFD").replace(/[\u0300-\u036f]/g, "").toUpperCase()
  ) ?? displayedCreator;
```

In this code, `normalize("NFD")` splits each accented character into its ASCII base and a diacritic, then `replace(/[\u0300-\u036f]/g, "")` removes the diacritics (which are in

the Unicode range 0300-036F). The `toUpperCase()` and `startsWith(firstLetter)` work as before.

Note that this will change the matching behavior for all names, not just names starting with "É". If you want to maintain the original behavior for names starting with ASCII characters, you would need to add a separate condition for that case.

I wonder if the replace step is actually necessary here... If that range of code points only includes combining characters that modify the character before them, then the decomposition alone should still start with the bare letter... I think?

You are correct in your thinking. The `normalize("NFD")` function decomposes composed characters into their base characters and combining characters. The base character for "É" is "E", so if you call `"É".normalize("NFD")`, you will get "E" followed by a combining acute accent.

Given this behavior, when you call `startsWith("E")` on the decomposed string, it will indeed return `true`. So, you can skip the `replace` step if all you need to do is a `startsWith` check:

js

 Copy code

```
const firstLetter = this.selectedCreatorFilter.toUpperCase();

displayedCreator =
  this.model.creators.find(creator =>
    creator.normalize("NFD").toUpperCase().startsWith(firstLetter)
  ) ?? displayedCreator;
```

However, be aware that if you need to do other string comparisons or operations, the combining characters may cause unexpected results. If you need to do more than just a

startswith check, you may want to remove the combining characters to avoid potential issues.